```
// Forge-hardened: process a refund for a merchant order
// Original logic preserved. 4 production-readiness issues fixed below.
const db = require('./db');
const psp = require('./payment-provider');

async function withRetry(fn, { attempts = 3, baseDelayMs = 200 } = {}) {
  let lastErr;
  for (let i = 0; i < attempts; i++) {
    try {
      return await fn();
    } catch (err) {
      lastErr = err;
      // Only retry on transient/network-shaped failures, not business-logic erro
      if (!err.isTransient) throw err;
      await new Promise((r) => setTimeout(r, baseDelayMs * 2 ** i));
    }
  }
  throw lastErr;
}

async function processRefund(orderId, amount) {
  const conn = await db.getConnection();

  try {
    const order = await conn.query('SELECT * FROM orders WHERE id = ?', [orderId]
    if (!order) {
      throw new Error('Order not found');
    }

    // FIX (unguarded irreversible action): idempotency check before calling the
    // Prevents duplicate refunds from retried requests, duplicate webhooks, or d
    const existingRefund = await conn.query(
      'SELECT * FROM refunds WHERE order_id = ?',
      [orderId]
    );
    if (existingRefund) {
      return { success: true, refId: existingRefund.psp_ref, alreadyProcessed: tr
    }

    // FIX (missing defensive handling): retry/backoff around the external PSP ca
    // Distinguishes "definitely failed" from "response lost, unknown state" inst
    // of assuming failure means nothing happened.
    const result = await withRetry(() => psp.refund({ orderId, amount }));
```

```
    await conn.query(
      'INSERT INTO refunds (order_id, amount, psp_ref) VALUES (?, ?, ?)',
      [orderId, amount, result.refId]
    );

    // FIX (environment drift): structured logger instead of a hardcoded /tmp pat
    // Works identically across local, containerized, and autoscaled environments
    logger.info('refund_processed', { orderId, amount, refId: result.refId });

    return { success: true, refId: result.refId };
  } finally {
    // FIX (resource exhaustion): connection is always released, success or failu
    conn.release();
  }
}

module.exports = { processRefund };
```